#C Parse the response JSON or throw an error if unsuccessful

The Fetch API is designed to be asynchronous, so rather than returning the result of the REST call directly it instead returns a Promise object, which can be used to register functions to be called when the operation completes. You don't need to worry about the details of that for this example, but just be aware that everything within the `.then(response => …)` section is executed if the request completed successfully, whereas everything in the `.catch(error => …)` section is executed if a network error occurs. If the request succeeds, then parse the response as JSON and log the details to the JavaScript console. Otherwise, any error is also logged to the console. The `response.ok` field indicates whether the HTTP status code was in the range 200-299, because these indicate successful responses in HTTP.

Create a new file called `natter.html` under `src/main/resources/public`, alongside the `natter.js` file you just created. Copy in the HTML from listing 4.2 and click save. The HTML includes the `natter.js` script you just created and displays the simple HTML form with fields for typing the space name and owner of the new space to be created. You can style the form with CSS if you want to make it a bit less ugly. The CSS in the listing just ensures that each form field is on a new line by filling up all remaining space with a large margin.

Listing 4.2 The Natter UI HTML

```
<!DOCTYPE html>
```